Crucible — Portal & Web Architecture

> *Client-facing Portal layer plus the full Forge UI layer. Covers: React component trees, Zustand stores, WebSocket management, auth/roles, report download. Stack: React 19 + TypeScript, Zustand, Recharts, Tailwind CSS, React Router v7. Deployment: Vercel (frontend) — same pattern as Hormuz sim. Implementation begins Week 5 (Portal) / Week 4 (Forge UI).*

Overview

Crucible has two user-facing layers built from a single React app with role-gated routes.

```
┌──────────────────────────────────────────────────────────────────────┐
│                       CRUCIBLE WEB APP (React 19)                    │
│                                                                      │
│  ┌────────────────────────┐    ┌──────────────────────────────────┐  │
│  │  FORGE (internal)      │    │  PORTAL (client-facing)          │  │
│  │                        │    │                                  │  │
│  │  /forge/sessions/:id   │    │  /portal/:simId                  │  │
│  │  /simulations/:id      │    │                                  │  │
│  │  /ensembles/:id        │    │  Clean, read-only executive view │  │
│  │                        │    │  Narrative feed                  │  │
│  │  Scoping agent chat    │    │  Fan charts (p10/p50/p90)        │  │
│  │  Live tick stream      │    │  Snapshot comparison table       │  │
│  │  Shock injection       │    │  Report download                 │  │
│  │  Snapshot management   │    │                                  │  │
│  │  Ensemble runner       │    │  No controls, no model internals │  │
│  └────────────────────────┘    └──────────────────────────────────┘  │
│                                                                      │
│  Shared: auth, API client, chart primitives, Zustand stores          │
└──────────────────────────────────────────────────────────────────────┘
```

---

1. Page Routing Tree

```
<App>
  <AuthProvider>                      — JWT decode, role injection
    <Router>
      /login                          → <LoginPage>
      /                               → redirect based on role
      |
      ├── [ADMIN | CONSULTANT routes]
      |     <ForgeLayout>             — sidebar nav, org/project context
      |     |
      |     ├── /forge/sessions/new   → <ForgeNewPage>      (start scoping)
      |     ├── /forge/sessions/:id   → <ForgePage>         (active intake)
```

```
    │     ├── /simulations          → <SimulationListPage>
    │     ├── /simulations/:id      → <SimulationPage>    (live dashboard)
    │     └── /ensembles/:id        → <EnsemblePage>      (fan chart + compare)
    │
    └── [CLIENT routes]
          <PortalLayout>            — minimal chrome, client branding
          │
          ├── /portal               → <PortalListPage>    (simulations shared with client)
          └── /portal/:simId        → <PortalPage>        (read-only sim view)
```

---

2. Auth Model

JWT claims

```typescript
// web/src/types/auth.ts

export type UserRole = "ADMIN" | "CONSULTANT" | "CLIENT";

export interface JWTClaims {
  sub: string;           // user_id
  email: string;
  role: UserRole;
  org_id: string;        // tenant — all data scoped under this
  project_ids?: string[]; // CLIENT: list of projects they can access
  sim_ids?: string[];    // CLIENT: list of simulations shared with them
  exp: number;
  iat: number;
}
```

Role access matrix

```
┌──────────────────────────┬───────┬────────────┬────────┐
│ Resource                 │ ADMIN │ CONSULTANT │ CLIENT │
├──────────────────────────┼───────┼────────────┼────────┤
│ All orgs / all sims      │  ✓    │            │        │
│ Own org — all sims       │  ✓    │     ✓      │        │
│ Shared sims only         │  ✓    │     ✓      │   ✓    │
│ Forge sessions (create)  │  ✓    │     ✓      │        │
│ Forge sessions (view)    │  ✓    │     ✓      │        │
│ SimSpec / model params   │  ✓    │     ✓      │        │
│ Raw env floats [0,1]     │  ✓    │     ✓      │        │
│ Display-value metrics    │  ✓    │     ✓      │   ✓    │
│ Narrative feed           │  ✓    │     ✓      │   ✓    │
│ Snapshot labels + compare │  ✓    │     ✓      │   ✓    │
│ Shock injection          │  ✓    │     ✓      │        │
│ Start / pause / resume sim │  ✓    │     ✓      │        │
│ Generate report          │  ✓    │     ✓      │   ✓    │
│ Download report          │  ✓    │     ✓      │   ✓    │
│ Approve calibration      │  ✓    │     ✓      │        │
```

```
│ Share sim with client       │  ✓    │     ✓      │        │
└─────────────────────────────┴───────┴────────────┴────────┘
```

Route guard middleware

```
// web/src/components/auth/RoleGuard.tsx

interface RoleGuardProps {
  allow: UserRole[];
  children: React.ReactNode;
  fallback?: React.ReactNode; // default: redirect to /login
}

// Usage:
// <RoleGuard allow={["ADMIN", "CONSULTANT"]}>
//   <SimulationPage />
// </RoleGuard>

// SimGuard additionally checks sim_ids claim for CLIENT role.
// If CLIENT tries to access a sim not in their claim list → 403 page.

interface SimGuardProps {
  simId: string;
  children: React.ReactNode;
}
```

Auth middleware pattern

```
Request → useAuthStore.getToken()
  → attach Bearer header via axios interceptor
  → 401 response → useAuthStore.logout() → redirect /login
  → 403 response → show AccessDeniedPage (do not expose resource exists)

CLIENT sim access check (client-side):
  const { claims } = useAuthStore();
  const allowed = claims.role !== "CLIENT" || claims.sim_ids?.includes(simId);
  if (!allowed) navigate("/403");
```

---

3. TypeScript Data Types

```
// web/src/types/sim.ts

// ── Simulation status ──────────────────────────────────────────

export type SimState =
  | "CREATED"
  | "CONFIGURING"
  | "CONFIGURED"
  | "RUNNING"
  | "PAUSED"
  | "COMPLETED"
  | "ARCHIVED";
```

```typescript
export interface SimStatus {
  sim_id: string;
  spec_id: string;
  state: SimState;
  tick: number;
  created_at: string;   // ISO 8601
  completed_at: string | null;
  error: string | null;
}

// ── Snapshot ──────────────────────────────────────────────────────

export interface SnapshotSummary {
  snapshot_id: string;
  sim_id: string;
  tick: number;
  label: string;
  timestamp: string;    // ISO 8601
}

export interface SnapshotDetail extends SnapshotSummary {
  env: Record<string, EnvDisplayValue>;   // display-annotated env
  agent_states: Record<string, unknown>;
  theory_states: Record<string, unknown>;
}

export interface EnvDisplayValue {
  normalized: number;   // raw [0,1] float from SimRunner
  display: number;      // normalized * scale
  unit: string;         // "billion USD", "index", etc.
  display_name: string;
}

// ── Metrics ───────────────────────────────────────────────────────

export interface MetricPoint {
  tick: number;
  value: number;        // display-scaled value
}

export interface MetricSeries {
  metric_id: string;
  env_key: string;
  display_name: string;
  unit: string;
  points: MetricPoint[];
}

// ── Narrative ─────────────────────────────────────────────────────

export interface NarrativeEntry {
  entry_id: string;
  sim_id: string;
  tick: number;
  content: string;      // plain English from NarrativeAgent (Claude)
  triggered_by: string | null;  // "threshold_X" | "snapshot_Y" | null
  created_at: string;
}

// ── Ensemble ──────────────────────────────────────────────────────

export interface PercentileBand {
  tick: number;
  p10: number;
  p50: number;
  p90: number;
}

export interface EnsembleResult {
  ensemble_id: string;
```

```
  sim_id: string;
  n_runs: number;
  state: "PENDING" | "RUNNING" | "COMPLETE" | "FAILED";
  progress: number;     // 0.0–1.0
  distributions: Record<string, number[]>;       // env_key → array of final values
  percentile_bands: Record<string, PercentileBand[]>; // env_key → time series
  wasserstein_distances?: Record<string, number>; // vs comparison ensemble
  completed_at: string | null;
}

// ── Reports ──────────────────────────────────────────────────────────

export type ReportFormat = "pdf" | "markdown" | "json";
export type ReportState = "PENDING" | "GENERATING" | "COMPLETE" | "FAILED";

export interface ReportJob {
  report_id: string;
  sim_id: string;
  format: ReportFormat;
  state: ReportState;
  snapshot_ids: string[];
  metric_keys: string[];
  download_url: string | null;
  created_at: string;
  completed_at: string | null;
}

// ── Forge session ────────────────────────────────────────────────────

export type ForgeState =
  | "intake"
  | "parallel_research"
  | "dynamic_interview"
  | "theory_mapping"
  | "validation"
  | "complete";

export interface ForgeMessage {
  role: "user" | "assistant" | "system" | "research";
  content: string;
  timestamp: string;
  research_source?: string; // "arxiv" | "ssrn" | "fred" | "worldbank" | "news"
}

export interface PartialSimSpec {
  name?: string;
  domain?: string;
  actors?: unknown[];
  theories?: unknown[];
  env_keys?: unknown[];
  // grows as scoping agent fills fields
  [key: string]: unknown;
}
```

---

4. Zustand Stores

useAuthStore

```
// web/src/stores/useAuthStore.ts

import { create } from "zustand";
```

```
import { persist } from "zustand/middleware";
import type { JWTClaims, UserRole } from "@/types/auth";

interface AuthState {
  token: string | null;
  claims: JWTClaims | null;
  role: UserRole | null;

  // actions
  login: (token: string) => void;          // decode + store JWT
  logout: () => void;                      // clear token, redirect /login
  getToken: () => string | null;
  isAllowed: (roles: UserRole[]) => boolean;
  canAccessSim: (simId: string) => boolean; // CLIENT role check
}

export const useAuthStore = create<AuthState>()(
  persist(
    (set, get) => ({
      token: null,
      claims: null,
      role: null,
      login: (token) => { /* decode JWT, set claims */ },
      logout: () => set({ token: null, claims: null, role: null }),
      getToken: () => get().token,
      isAllowed: (roles) => roles.includes(get().role ?? "CLIENT"),
      canAccessSim: (simId) => {
        const { claims } = get();
        if (!claims) return false;
        if (claims.role !== "CLIENT") return true;
        return claims.sim_ids?.includes(simId) ?? false;
      },
    }),
    { name: "crucible-auth" }
  )
);
```

useSimStore

```
// web/src/stores/useSimStore.ts

import { create } from "zustand";
import type { SimStatus, SnapshotSummary, MetricSeries } from "@/types/sim";

interface SimState {
  status: SimStatus | null;
  snapshots: SnapshotSummary[];
  metrics: MetricSeries[];
  wsConnected: boolean;
  wsError: string | null;

  // actions
  setStatus: (s: SimStatus) => void;
  applyTickEvent: (event: WsTickEvent) => void;  // live tick from WS
  setSnapshots: (snaps: SnapshotSummary[]) => void;
  addSnapshot: (snap: SnapshotSummary) => void;
  setMetrics: (metrics: MetricSeries[]) => void;
  appendMetricPoint: (envKey: string, tick: number, value: number) => void;
  setWsStatus: (connected: boolean, error?: string) => void;
  reset: () => void;
}

// WsTickEvent matches the WebSocket message schema from the API
export interface WsTickEvent {
  type: "tick";
  tick: number;
```

```
  env: Record<string, number>;           // raw normalized floats
  metrics: Record<string, number>;       // display-scaled, keyed by env_key
  narrative_entries?: NarrativeEntry[];  // entries triggered this tick
}

export const useSimStore = create<SimState>()((set, get) => ({
  status: null,
  snapshots: [],
  metrics: [],
  wsConnected: false,
  wsError: null,
  setStatus: (status) => set({ status }),
  applyTickEvent: (event) => {
    set((s) => ({ status: s.status ? { ...s.status, tick: event.tick } : s.status }));
    // append metric points for all keys in event.metrics
  },
  setSnapshots: (snapshots) => set({ snapshots }),
  addSnapshot: (snap) => set((s) => ({ snapshots: [...s.snapshots, snap] })),
  setMetrics: (metrics) => set({ metrics }),
  appendMetricPoint: (envKey, tick, value) => { /* update metrics array */ },
  setWsStatus: (connected, error) => set({ wsConnected: connected, wsError: error ?? null }),
  reset: () => set({ status: null, snapshots: [], metrics: [], wsConnected: false }),
}));
```

useEnsembleStore

```
// web/src/stores/useEnsembleStore.ts

import { create } from "zustand";
import type { EnsembleResult } from "@/types/sim";

interface EnsembleState {
  results: Record<string, EnsembleResult>;  // keyed by ensemble_id
  activeId: string | null;
  compareId: string | null;                 // second ensemble for side-by-side
  wsConnected: boolean;

  // actions
  setResult: (r: EnsembleResult) => void;
  updateProgress: (ensembleId: string, progress: number, partial?: Partial<EnsembleResult>) => void;
  setActiveId: (id: string) => void;
  setCompareId: (id: string | null) => void;
  setWsConnected: (v: boolean) => void;
}

export const useEnsembleStore = create<EnsembleState>()((set) => ({
  results: {},
  activeId: null,
  compareId: null,
  wsConnected: false,
  setResult: (r) => set((s) => ({ results: { ...s.results, [r.ensemble_id]: r } })),
  updateProgress: (id, progress, partial) =>
    set((s) => ({
      results: {
        ...s.results,
        [id]: { ...s.results[id], progress, ...(partial ?? {}) },
      },
    })),
  setActiveId: (id) => set({ activeId: id }),
  setCompareId: (id) => set({ compareId: id }),
  setWsConnected: (v) => set({ wsConnected: v }),
}));
```

useNarrativeStore

```
// web/src/stores/useNarrativeStore.ts

import { create } from "zustand";
import type { NarrativeEntry } from "@/types/sim";

interface NarrativeState {
  entries: NarrativeEntry[];
  unreadCount: number;          // for Portal badge / scroll indicator

  // actions
  setEntries: (entries: NarrativeEntry[]) => void;
  addEntry: (entry: NarrativeEntry) => void;   // called on WS tick push
  markAllRead: () => void;
  reset: () => void;
}

export const useNarrativeStore = create<NarrativeState>()((set) => ({
  entries: [],
  unreadCount: 0,
  setEntries: (entries) => set({ entries, unreadCount: 0 }),
  addEntry: (entry) =>
    set((s) => ({ entries: [...s.entries, entry], unreadCount: s.unreadCount + 1 })),
  markAllRead: () => set({ unreadCount: 0 }),
  reset: () => set({ entries: [], unreadCount: 0 }),
}));
```

useForgeStore

```
// web/src/stores/useForgeStore.ts

import { create } from "zustand";
import type { ForgeMessage, ForgeState, PartialSimSpec } from "@/types/sim";

interface ForgeStoreState {
  sessionId: string | null;
  state: ForgeState | null;
  conversation: ForgeMessage[];
  partialSpec: PartialSimSpec;
  researchStreaming: boolean;   // true while background research is live
  wsConnected: boolean;
  completedSimId: string | null;  // set when state === "complete"

  // actions
  setSession: (id: string) => void;
  setState: (s: ForgeState) => void;
  addMessage: (msg: ForgeMessage) => void;
  updatePartialSpec: (patch: Partial<PartialSimSpec>) => void;
  setResearchStreaming: (v: boolean) => void;
  setWsConnected: (v: boolean) => void;
  setCompletedSimId: (id: string) => void;
  reset: () => void;
}

export const useForgeStore = create<ForgeStoreState>()((set) => ({
  sessionId: null,
  state: null,
  conversation: [],
  partialSpec: {},
  researchStreaming: false,
  wsConnected: false,
  completedSimId: null,
  setSession: (id) => set({ sessionId: id }),
```

```
  setState: (s) => set({ state: s }),
  addMessage: (msg) => set((st) => ({ conversation: [...st.conversation, msg] })),
  updatePartialSpec: (patch) => set((st) => ({ partialSpec: { ...st.partialSpec, ...patch } })),
  setResearchStreaming: (v) => set({ researchStreaming: v }),
  setWsConnected: (v) => set({ wsConnected: v }),
  setCompletedSimId: (id) => set({ completedSimId: id }),
  reset: () => set({ sessionId: null, state: null, conversation: [], partialSpec: {} }),
}));
```

---

5. WebSocket Management

Connection lifecycle

```
useSimWs(simId)   →  opens WS /api/simulations/{id}/stream
useEnsembleWs(id) →  opens WS /api/ensembles/{id}/stream
useForgeWs(id)    →  opens WS /forge/sessions/{id}/stream
```

All three hooks follow the same pattern, implemented in `web/src/hooks/useWebSocket.ts`.

WebSocket message flow

```
Browser                              FastAPI WS endpoint
  |                                        |
  |── connect ──────────────────────────→ |
  |                                        |  subscribe to SimEventBus
  |← {"type":"connected","tick":42} ────  |
  |                                        |
  |                    (SimRunner ticks)   |
  |← {"type":"tick", ...WsTickEvent} ──── |  every tick
  |                                        |
  |← {"type":"snapshot_taken", ...} ────── |  on snapshot event
  |                                        |
  |← {"type":"narrative", ...entry} ────── |  when NarrativeAgent fires
  |                                        |
  |← {"type":"paused"} ──────────────────  |  on pause
  |← {"type":"completed","tick":N} ──────  |  sim finished
  |                                        |
  |── close ──────────────────────────→ |  on unmount / nav away
```

WS event types

```
// web/src/types/ws.ts

export type WsEventType =
  | "connected"
  | "tick"
  | "snapshot_taken"
  | "narrative"
  | "shock_applied"
  | "paused"
```

```
  | "resumed"
  | "completed"
  | "error"
  // Forge session events
  | "research_result"
  | "spec_update"
  | "forge_state_change"
  // Ensemble events
  | "ensemble_progress"
  | "ensemble_complete";

export type WsEvent =
  | { type: "connected"; tick: number }
  | { type: "tick" } & WsTickEvent
  | { type: "snapshot_taken"; snapshot: SnapshotSummary }
  | { type: "narrative"; entry: NarrativeEntry }
  | { type: "paused"; tick: number }
  | { type: "completed"; tick: number }
  | { type: "research_result"; source: string; summary: string; spec_patch: Partial<PartialSimSpec> }
  | { type: "spec_update"; patch: Partial<PartialSimSpec> }
  | { type: "forge_state_change"; state: ForgeState }
  | { type: "ensemble_progress"; progress: number; partial: Partial<EnsembleResult> }
  | { type: "ensemble_complete"; result: EnsembleResult }
  | { type: "error"; message: string };
```

`useWebSocket` base hook

```
// web/src/hooks/useWebSocket.ts

interface UseWebSocketOptions<T> {
  url: string;
  onMessage: (event: T) => void;
  onOpen?: () => void;
  onClose?: () => void;
  enabled?: boolean;        // default true — set false to defer opening
  reconnectDelayMs?: number; // default 2000, doubles on each retry (max 30s)
  maxRetries?: number;       // default 10; null = unlimited
}

// Returns { connected, error, send, close }
// Reconnect logic: exponential backoff, stops on sim COMPLETED or unmount.
// On "completed" message, marks internal flag — no more reconnects.
```

Message routing

```
// web/src/hooks/useSimWs.ts
// Thin wrapper over useWebSocket that routes events to useSimStore + useNarrativeStore.

export function useSimWs(simId: string, enabled = true) {
  const { applyTickEvent, setWsStatus, addSnapshot } = useSimStore();
  const { addEntry } = useNarrativeStore();

  return useWebSocket<WsEvent>({
    url: `/api/simulations/${simId}/stream`,
    enabled,
    onMessage: (event) => {
      switch (event.type) {
        case "tick":           return applyTickEvent(event);
        case "snapshot_taken": return addSnapshot(event.snapshot);
        case "narrative":      return addEntry(event.entry);
        case "completed":      return setWsStatus(false); // stop reconnect
        default:               break;
```

```
      }
    },
    onOpen:  () => setWsStatus(true),
    onClose: () => setWsStatus(false),
  });
}
```

Reconnect rules

Condition	Behaviour
Normal disconnect (network blip)	Exponential backoff: 2s, 4s, 8s … max 30s
Server sends `{"type":"completed"}`	No reconnect — sim is done
Component unmounts	Close immediately, no reconnect
10 consecutive failures	Stop, set `wsError`, show error banner
User navigates away	Close, clear store via `reset()`

---

6. Forge UI

6.1 ForgePage — scoping agent chat

```
/forge/sessions/:sessionId
```

```
<ForgePage>
  ├── <ForgeHeader>              — session title, state badge, progress indicator
  │
  ├── [left panel — 55% width]
  │   ├── <ForgeChat>            — conversation thread (messages)
  │   │   ├── <ForgeChatMessage role="user">
  │   │   ├── <ForgeChatMessage role="assistant">
  │   │   └── <ResearchResultCard source="arxiv" summary="…">
  │   │
  │   ├── <ResearchStreamIndicator>  — "Searching arXiv… FRED… World Bank…"
  │   │                               animated while researchStreaming === true
  │   └── <ForgeChatInput>          — textarea + send, disabled during research
  │
  └── [right panel — 45% width]
      └── <SpecPreview>             — live SimSpec as it fills in
          ├── <SpecSection title="Domain">
          ├── <SpecSection title="Actors">
          ├── <SpecSection title="Theories">
          ├── <SpecSection title="Environment Keys">
          └── <SpecReadyBanner>      — appears when state === "complete"
                                       "Simulation ready — Launch →"
```

Component signatures

```
// web/src/pages/ForgePage/ForgeChat.tsx
/** Full conversation thread. Subscribes to useForgeStore.conversation. */
interface ForgeChatProps {
  sessionId: string;
  className?: string;
}
export function ForgeChat({ sessionId, className }: ForgeChatProps): JSX.Element

// web/src/pages/ForgePage/ResearchResultCard.tsx
/** Collapsible card showing one research result event from the WS stream. */
interface ResearchResultCardProps {
  source: string;       // "arxiv" | "ssrn" | "fred" | "worldbank" | "news"
  summary: string;
  specPatch?: Partial<PartialSimSpec>;
  timestamp: string;
}
export function ResearchResultCard(props: ResearchResultCardProps): JSX.Element

// web/src/pages/ForgePage/SpecPreview.tsx
/** Right panel: live view of the growing SimSpec. Sections fade in as fields populate. */
interface SpecPreviewProps {
  partialSpec: PartialSimSpec;
  state: ForgeState;
  onLaunch?: (simId: string) => void;  // fired after state === "complete"
}
export function SpecPreview(props: SpecPreviewProps): JSX.Element
```

6.2 SimulationPage — consultant dashboard

```
/simulations/:simId
```

```
<SimulationPage>
  ├── <SimulationHeader>
  │   ├── sim name, tick counter, state badge
  │   └── <SimControls>             — Start / Pause / Resume buttons
  │                                   POST /api/simulations/{id}/control
  │
  ├── [top row]
  │   └── <SimEnvironmentPanel>     — current env state (display values)
  │       └── <EnvKeyCard key="…">  — one card per env key with display_name + unit
  │
  ├── [middle row — charts]
  │   └── <SimulationMetricsChart   — Recharts LineChart
  │         metrics={MetricSeries[]}
  │         highlightTick={currentTick}
  │       />
  │
  ├── [lower-left — snapshots]
  │   ├── <SnapshotList>
  │   │   └── <SnapshotRow> × N     — label, tick, "Compare" checkbox
  │   ├── <TakeSnapshotButton>      — opens <SnapshotLabelModal>
  │   └── <SnapshotCompareTable     — appears when 2 snapshots selected
  │         snapshotA={SnapshotDetail}
  │         snapshotB={SnapshotDetail}
  │       />
  │
```

```
  ├── [lower-middle — shocks]
  │   └── <ShockInjector>           — env key selector + magnitude slider
  │                                   POST /api/simulations/{id}/shocks
  │
  └── [lower-right — links]
      └── <EnsembleLaunchButton>    — opens EnsembleConfigModal
                                      POST /api/simulations/{id}/ensemble
```

Component signatures

```typescript
// web/src/pages/SimulationPage/SimulationMetricsChart.tsx
/** Multi-series line chart with Recharts. Shows display-scaled metric time series. */
interface SimulationMetricsChartProps {
  metrics: MetricSeries[];
  highlightTick?: number;  // vertical reference line for "now"
  height?: number;
  className?: string;
}
export function SimulationMetricsChart(props: SimulationMetricsChartProps): JSX.Element

// web/src/pages/SimulationPage/SnapshotCompareTable.tsx
/** Side-by-side env value comparison for two named snapshots. */
interface SnapshotCompareTableProps {
  snapshotA: SnapshotDetail;
  snapshotB: SnapshotDetail;
  highlightDelta?: boolean;  // colour cells where |deltaA-B| > threshold
}
export function SnapshotCompareTable(props: SnapshotCompareTableProps): JSX.Element

// web/src/pages/SimulationPage/ShockInjector.tsx
/** Form to inject a manual shock. Selects env key, magnitude, description. */
interface ShockInjectorProps {
  simId: string;
  envKeys: string[];
  onShockApplied?: () => void;
}
export function ShockInjector(props: ShockInjectorProps): JSX.Element
```

---

7. Portal UI

PortalPage — client-facing view

```
/portal/:simId
```

The Portal is intentionally minimal: no model internals, no controls, no raw floats. It presents a polished executive view.

```
<PortalPage>
  ├── <PortalHeader>                — sim name, client org logo, last updated
  │
  ├── <PortalStatusBanner>          — "Simulation running — Day 23 of scenario"
  │                                   or "Analysis complete as of March 15, 2026"
  │
```

```
  ├── [section: Narrative Feed]
  │   └── <NarrativeFeed            — chronological plain-English entries
  │         entries={NarrativeEntry[]}
  │         maxVisible={10}
  │         expandable
  │       />
  │       └── <NarrativeEntryCard>  — tick badge, content, triggered-by tag
  │
  ├── [section: Outcome Metrics]
  │   └── <EnsemblePortalSection    — fan chart if ensemble exists
  │         simId={simId}           — falls back to single-run line chart
  │       />
  │       └── <EnsembleFanChart     — p10/p50/p90 bands, one per metric
  │             bands={PercentileBand[]}
  │             metricName="…"
  │             unit="…"
  │           />
  │
  ├── [section: Scenario Snapshots]
  │   └── <SnapshotCompareTable     — read-only, named snapshots chosen by consultant
  │         snapshotA={SnapshotDetail}
  │         snapshotB={SnapshotDetail}
  │         readOnly
  │       />
  │
  └── [section: Report]
      └── <ReportDownloadPanel      — format selector + download button
            simId={simId}
          />
```

Portal component hierarchy (ASCII)

```
PortalPage
├── PortalHeader
│   ├── OrgLogo
│   └── SimStatusBadge
├── PortalStatusBanner
├── NarrativeFeed
│   └── NarrativeEntryCard [× N]
│       ├── TickBadge
│       ├── EntryContent (prose)
│       └── TriggerTag ("Threshold crossed" | "Snapshot")
├── EnsemblePortalSection
│   ├── SectionHeading
│   ├── MetricSelector        (tab row — one tab per metric key)
│   └── EnsembleFanChart      (Recharts AreaChart, 3 areas)
│       ├── Area fill p10–p90 (light tint)
│       ├── Area fill p25–p75 (medium tint, if available)
│       └── Line  p50         (solid median)
├── SnapshotCompareTable
│   ├── SnapshotColumnHeader [× 2]
│   └── CompareRow [× N env keys]
```

```
│       ├── MetricLabel
│       ├── ValueCell (snapshotA)
│       └── ValueCell (snapshotB, delta badge)
└── ReportDownloadPanel
    ├── FormatToggle (PDF / Markdown / JSON)
    ├── GenerateButton
    ├── ProgressBar (while GENERATING)
    └── DownloadLink (when COMPLETE)
```

Component signatures

```typescript
// web/src/pages/PortalPage/NarrativeFeed.tsx
/**
 * Scrollable chronological list of NarrativeAgent entries.
 * Newest entries animate in at the bottom. Auto-scrolls when live.
 */
interface NarrativeFeedProps {
  entries: NarrativeEntry[];
  maxVisible?: number;   // default 20; "Show more" link below
  live?: boolean;        // true → auto-scroll to bottom on new entries
  className?: string;
}
export function NarrativeFeed(props: NarrativeFeedProps): JSX.Element

// web/src/pages/PortalPage/EnsembleFanChart.tsx
/**
 * Recharts AreaChart showing p10/p50/p90 percentile bands over time.
 * Used in both PortalPage (EnsemblePortalSection) and EnsemblePage.
 */
interface EnsembleFanChartProps {
  bands: PercentileBand[];
  metricName: string;
  unit: string;
  height?: number;
  thresholdValue?: number;          // optional horizontal reference line
  thresholdLabel?: string;          // e.g. "Critical threshold"
  className?: string;
}
export function EnsembleFanChart(props: EnsembleFanChartProps): JSX.Element

// web/src/pages/PortalPage/EnsemblePortalSection.tsx
/**
 * Fan chart section for Portal. Fetches ensemble result via GET /api/ensembles/{id}/results.
 * Falls back to single-run MetricSeries if no ensemble exists for this sim.
 */
interface EnsemblePortalSectionProps {
  simId: string;
  ensembleId?: string;    // if known; otherwise discovered from sim status
  className?: string;
}
export function EnsemblePortalSection(props: EnsemblePortalSectionProps): JSX.Element
```

---

8. EnsemblePage (Forge)

```
/ensembles/:ensembleId
```

Internal consultant view. Full ensemble analysis with scenario comparison.

```
<EnsemblePage>
  ├── <EnsembleHeader>              — ensemble ID, n_runs, sim name, state
  ├── <EnsembleProgressBar>         — shown while state === "RUNNING"
  │
  ├── [fan charts — one per metric key]
  │   └── <EnsembleFanChart>        — reused from Portal section
  │
  ├── [scenario comparison]
  │   ├── <CompareEnsembleSelector> — dropdown to pick a second ensemble_id
  │   ├── <WassersteinTable>        — distribution shift per env key
  │   │   └── one row per metric: W-distance + colour indicator
  │   └── <SideBySideFanChart>      — two EnsembleFanCharts, same axis scale
  │
  └── [threshold probability]
      └── <ThresholdProbabilityPanel>
          ├── <MetricSelector>
          ├── <ThresholdInput>      — user sets threshold value
          └── <ProbabilityReadout>  — "P(outcome > threshold) = 73%"
                                       computed from distributions array
```

Component signatures

```typescript
// web/src/pages/EnsemblePage/WassersteinTable.tsx
/**
 * Table showing Wasserstein (earth-mover) distance between two ensembles
 * for each metric. Cells are coloured: green (low shift) → red (high shift).
 */
interface WassersteinTableProps {
  ensembleA: EnsembleResult;
  ensembleB: EnsembleResult;
  metricKeys: string[];
}
export function WassersteinTable(props: WassersteinTableProps): JSX.Element

// web/src/pages/EnsemblePage/ThresholdProbabilityPanel.tsx
/**
 * Computes P(metric > threshold) from the distributions array (final-tick values).
 * All computation is client-side from the fetched EnsembleResult.
 */
interface ThresholdProbabilityPanelProps {
  ensemble: EnsembleResult;
}
export function ThresholdProbabilityPanel(props: ThresholdProbabilityPanelProps): JSX.Element
```

---

9. Report Download Flow

```
Consultant / Client clicks "Generate Report"
         │
         ▼
<ReportDownloadPanel> — opens <ReportConfigModal>
```

```
        │
        ├── Select format: PDF | Markdown | JSON
        ├── Select snapshots (checkbox list from useSimStore.snapshots)
        └── Select metric keys (checkbox list)
        │
        ▼ on confirm
POST /api/simulations/{id}/reports
  body: { format, snapshot_ids, metric_keys }
        │
        ▼
ReportJob { report_id, state: "PENDING" }
        │
        ▼ poll GET /api/reports/{id} every 3s
        │
        ├── state === "GENERATING" → show progress bar + spinner
        └── state === "COMPLETE"   → download_url available
                  │
                  ▼
             <a href={download_url} download> — trigger browser download
             (PDF: Content-Disposition attachment)
             (Markdown/JSON: same pattern)
```

Report polling hook

```ts
// web/src/hooks/useReportPoller.ts

interface UseReportPollerOptions {
  reportId: string | null;
  intervalMs?: number;       // default 3000
  onComplete: (job: ReportJob) => void;
  onError: (err: string) => void;
}

// Polls until state is COMPLETE or FAILED, then stops.
// Returns { job, polling }
export function useReportPoller(opts: UseReportPollerOptions): {
  job: ReportJob | null;
  polling: boolean;
}
```

`ReportDownloadPanel` component

```ts
// web/src/components/ReportDownloadPanel.tsx
/**
 * Self-contained report generation and download UI.
 * Used in both PortalPage (client) and SimulationPage (consultant).
 * In Portal (CLIENT role), snapshot/metric selection is hidden — consultant pre-selected them.
 */
interface ReportDownloadPanelProps {
  simId: string;
  availableSnapshots?: SnapshotSummary[];   // omitted for CLIENT role
  availableMetrics?: MetricSeries[];         // omitted for CLIENT role
  defaultSnapshotIds?: string[];            // pre-selected by consultant
  defaultMetricKeys?: string[];
  className?: string;
```

```
}
export function ReportDownloadPanel(props: ReportDownloadPanelProps): JSX.Element
```

---

10. Shared Component Library

All chart primitives, layout shells, and UI atoms live in `web/src/components/` and are imported by both Forge and Portal pages.

```
web/src/components/
├── auth/
│   ├── RoleGuard.tsx            — role-based route guard
│   └── SimGuard.tsx             — sim-id access check for CLIENT role
│
├── charts/
│   ├── EnsembleFanChart.tsx     — p10/p50/p90 AreaChart (Recharts)
│   ├── MetricsLineChart.tsx     — generic multi-series line chart
│   └── DistributionHistogram.tsx— final-value distribution histogram
│
├── sim/
│   ├── SimStatusBadge.tsx       — coloured pill: RUNNING / PAUSED / COMPLETE
│   ├── TickCounter.tsx          — animated tick number
│   ├── SnapshotCompareTable.tsx — shared by Forge + Portal
│   └── NarrativeFeed.tsx        — shared by Forge + Portal
│
├── report/
│   └── ReportDownloadPanel.tsx  — shared by Forge + Portal
│
└── ui/
    ├── SectionHeading.tsx
    ├── LoadingSpinner.tsx
    ├── ErrorBanner.tsx
    └── EmptyState.tsx
```

---

11. API Client

```
// web/src/api/client.ts
// Axios instance with JWT interceptor and base URL from VITE_API_URL env var.

// web/src/api/simulations.ts
export const api = {
  getSimStatus:     (id: string) => GET<SimStatus>(`/simulations/${id}`),
  getSnapshots:     (id: string) => GET<SnapshotSummary[]>(`/simulations/${id}/snapshots`),
  getSnapshotDetail:(id: string, snapId: string) => GET<SnapshotDetail>(`/simulations/${id}/snapshots/${snapI
d}`),
  getMetrics:       (id: string, params?: MetricQueryParams) => GET<MetricSeries[]>(`/simulations/${id}/metri
cs`, { params }),
  getNarrative:     (id: string) => GET<NarrativeEntry[]>(`/simulations/${id}/narrative`),
  control:          (id: string, action: "start"|"pause"|"resume") => POST(`/simulations/${id}/control`, { ac
tion }),
```

```
  injectShock:      (id: string, req: ShockRequest) => POST(`/simulations/${id}/shocks`, req),
  createReport:     (id: string, req: ReportRequest) => POST<ReportJob>(`/simulations/${id}/reports`, req),
  getReport:        (reportId: string) => GET<ReportJob>(`/reports/${reportId}`),
  getEnsemble:      (id: string) => GET<EnsembleResult>(`/ensembles/${id}/results`),
};
```

---

12. Key Design Decisions

Decision	Rationale
Single React app, two layouts	Share auth, stores, component library. Role-gated routes rather than two separate apps. Simpler deployment.
CLIENT access enforced client-side AND server-side	JWT claims carry `sim_ids`; API enforces same check. Client-side prevents UI from showing forbidden routes.
Portal is pure read — no WS connection	Clients get a polished static view. No live tick stream reduces complexity and avoids confusing half-finished ticks. Portal fetches on page load only.
`display_env()` on SimSpec, not on client	Raw `[0,1]` floats never reach Portal. API always returns display-annotated values. Portal never does unit conversion.
EnsembleFanChart shared by Portal and EnsemblePage	Same Recharts primitive, different context. Portal section fetches its own data; EnsemblePage passes from store.
Polling for report status (not WS)	Reports are a one-off job, not a streaming concern. 3s polling is adequate and simpler to implement.
Zustand over React Query for live sim state	WS events mutate store directly. React Query would require manual cache invalidation on every tick — Zustand is cleaner for push-based updates.
React Query for REST fetches (snapshots, metrics, narrative)	GET endpoints benefit from caching, deduplication, background refresh. No conflict with Zustand — stores hold the live WS state; React Query holds fetched snapshots.
Recharts over D3 directly	Consistent with Hormuz sim. React-native, good TypeScript types, Tailwind-compatible.
Vercel deployment (no Vercel CLI)	Same pattern as Hormuz sim. Always `git push` → auto-deploy. Never `vercel deploy` — creates duplicate projects.
`python -m uvicorn` in development	Uvicorn not on PATH in dev environment. Always use module invocation.

---

13. File Layout

```
web/
├── src/
│   ├── main.tsx
│   ├── App.tsx                  — router, AuthProvider wrapper
│   |
```

```
│   ├── types/
│   │   ├── auth.ts              — JWTClaims, UserRole
│   │   ├── sim.ts               — SimStatus, SnapshotSummary, MetricSeries, etc.
│   │   └── ws.ts                — WsEvent union type
│   │
│   ├── stores/
│   │   ├── useAuthStore.ts
│   │   ├── useSimStore.ts
│   │   ├── useEnsembleStore.ts
│   │   ├── useNarrativeStore.ts
│   │   └── useForgeStore.ts
│   │
│   ├── hooks/
│   │   ├── useWebSocket.ts      — base WS hook with reconnect
│   │   ├── useSimWs.ts          — sim tick stream → useSimStore
│   │   ├── useEnsembleWs.ts     — ensemble progress → useEnsembleStore
│   │   ├── useForgeWs.ts        — forge session stream → useForgeStore
│   │   └── useReportPoller.ts   — report job polling
│   │
│   ├── api/
│   │   ├── client.ts            — axios instance + interceptors
│   │   └── simulations.ts       — typed API functions
│   │
│   ├── components/              — shared primitives (see §10)
│   │
│   ├── layouts/
│   │   ├── ForgeLayout.tsx      — sidebar nav for ADMIN / CONSULTANT
│   │   └── PortalLayout.tsx     — minimal chrome for CLIENT
│   │
│   └── pages/
│       ├── LoginPage/
│       ├── ForgePage/           — §6.1: scoping agent chat
│       │   ├── index.tsx
│       │   ├── ForgeChat.tsx
│       │   ├── ForgeChatMessage.tsx
│       │   ├── ForgeChatInput.tsx
│       │   ├── ResearchResultCard.tsx
│       │   ├── ResearchStreamIndicator.tsx
│       │   └── SpecPreview.tsx
│       │
│       ├── SimulationPage/      — §6.2: consultant dashboard
│       │   ├── index.tsx
│       │   ├── SimulationHeader.tsx
│       │   ├── SimControls.tsx
│       │   ├── SimEnvironmentPanel.tsx
│       │   ├── EnvKeyCard.tsx
│       │   ├── SimulationMetricsChart.tsx
│       │   ├── SnapshotList.tsx
│       │   ├── SnapshotLabelModal.tsx
│       │   ├── ShockInjector.tsx
│       │   └── EnsembleLaunchButton.tsx
│       │
│       ├── EnsemblePage/        — §8: fan chart + comparison
│       │   ├── index.tsx
```

```
│       │   ├── EnsembleHeader.tsx
│       │   ├── EnsembleProgressBar.tsx
│       │   ├── CompareEnsembleSelector.tsx
│       │   ├── WassersteinTable.tsx
│       │   ├── SideBySideFanChart.tsx
│       │   └── ThresholdProbabilityPanel.tsx
│       │
│       ├── PortalListPage/       — list of sims shared with client
│       │
│       └── PortalPage/           — §7: client-facing view
│           ├── index.tsx
│           ├── PortalHeader.tsx
│           ├── PortalStatusBanner.tsx
│           ├── EnsemblePortalSection.tsx
│           └── ReportConfigModal.tsx
│
├── .env.local                    — VITE_API_URL=http://localhost:8000
├── vite.config.ts
├── tailwind.config.ts
└── tsconfig.json
```